Assessing the Integrity of Digital Evidence: A Low-Latency Forensic Approach to Deepfake Detection

by

Joel Villalta–Corcino

Christopher Swasey

A Capstone Project

Submitted in Partial Fulfillment of the Requirements for the Degree of

Master of Science in Computer Information Technology

Central Connecticut State University

New Britain, Connecticut

December, 2025

Advisor: Dr. Ramyapandian Vijayakanthan

Abstract

Deepfake videos pose a growing challenge to the integrity of digital evidence and online trust. While most detectors operate in the spatial domain and rely on pixel-level artifacts that can disappear as generative models improve, this capstone explores a frequency-domain approach explicitly optimized for low-latency forensic triage. We implement a three-stage pipeline: (1) a baseline 3D convolutional neural network (3D CNN) trained on short RGB video clips, (2) an intermediate ResNet50 model that leverages log-magnitude FFT spectra via transfer learning, and (3) a final fine-tuned ResNet50 adapted to frequency-domain features and evaluated for inference latency.

Experiments on the FaceForensics++ dataset show a clear progression in performance. The baseline 3D CNN achieves test accuracies of 57–66% with AUC values between 0.55 and 0.74, indicating limited separability between real and fake videos. The intermediate FFT-based ResNet50 improves accuracy to 72–76% with AUC values around 0.82–0.85. After fine-tuning the top convolutional block, the final frequency-domain model reaches 90–92% accuracy with AUC values exceeding 0.95 across six independent runs. Latency measurements on an NVIDIA Tesla T4 GPU show median per-frame inference times of roughly 2.7,ms for the final model, supporting near-real-time triage of suspicious content.

Taken together, these results demonstrate that frequency-domain representations, combined with transfer learning and light fine-tuning, can deliver both high accuracy and low latency, making them well-suited for integration into digital forensic workflows where rapid, reliable screening of video evidence is essential.

Acknowledgments

We would like to thank our advisor, Dr. Ramyapandian Vijayakanthan, for guiding this project from the initial proposal through the final experiments. We also appreciate the feedback from our classmates and faculty in the Computer Information Technology program at Central Connecticut State University. Finally, we acknowledge the maintainers of the FaceForensics++ dataset and the developers of PyTorch and Google Colab, whose tools made this work feasible.

Contents

1 Introduction **8**

2 Background and Related Work **11**

2.1 Deepfake Generation and Threats 11

2.2 Pixel-Domain Detection 11

2.3 Frequency-Domain Approaches 12

3 Methodology **13**

3.1 Data and Experimental Setup 13

3.2 Data Loading and Frame Extraction 14

3.3 Baseline Model: 3D CNN in the Pixel Domain 15

3.4 Frequency-Domain Preprocessing 15

3.5 Intermediate Model: ResNet50 with Transfer Learning 16

3.6 Final Model: Fine-Tuning ResNet50 17

3.7 Implementation in Google Colab 18

3.8 Evaluation Metrics 19

4 Results **20**

4.1 Baseline 3D CNN Performance 20

4.2 Intermediate Frequency-Domain Model 21

4.3 Final Fine-Tuned Model 22

4.4 Latency Measurements 23

4.5 Summary of Improvements Across Stages 24

4.6 Cross-Run Accuracy and AUC Statistics 25

5 Discussion **26**

5.1 Why the Frequency Domain Helps 26

5.2 Overfitting and Generalization 26

5.3 Forensic Utility 27

6 Limitations and Future Work **27**

7 Conclusion **28**

A Additional Results and Per-Run Metrics **29**

B Complete Classification Reports **31**

C Additional Figures **34**

C.1 Example FFT Spectra and Training Curves 34

C.2 Baseline Model Confusion Matrices by Run 35

C.3 Baseline Model ROC Curves by Run 36

C.4 Intermediate Model Confusion Matrices by Run 37

C.5 Intermediate Model ROC Curves by Run 37

C.6 Final Model Confusion Matrices by Run 38

C.7 Final Model ROC Curves by Run 39

D Google Colab Notebook Structure **40**

E Biographical Note **44**

List of Figures

1 High-level deepfake detection pipeline showing the baseline pixel-domain 3D CNN, the FFT-based ResNet model, and the fine-tuned final model. 10

2 End-to-end deepfake detection pipeline: from raw videos to frame extraction, FFT conversion, model inference, and forensic reporting. . . 14

3 From video frames to frequency-domain representation: each RGB frame is converted to grayscale, transformed with a 2D FFT, shifted to center low frequencies, and mapped to a log-magnitude spectrum saved as an image. 16

4 Google Colab notebook structure for the FFT-based deepfake detector, showing data loading, baseline training, FFT preprocessing, transfer learning, fine-tuning, evaluation, and latency measurement cells. . . . 18

5 Baseline pixel-domain model performance on the test set. 21

6 Intermediate frequency-domain model performance on the FFT image test set. 22

7 Final frequency-domain model performance on the test set. 23

8 Performance comparison across the three stages of the pipeline. . . . 25

9 Example FFT magnitude spectra for representative real and fake frames, illustrating characteristic frequency-domain patterns. 34

10 Training and validation loss curves for the FFT-based ResNet50 model (Run 6 intermediate stage) across 30 epochs. 35

11 Baseline model confusion matrices for Runs 1–3. 35

12 Baseline model confusion matrices for Runs 4–6. 36

13 Baseline model ROC curves for Runs 1–3. 36

14 Baseline model ROC curves for Runs 4–6. 36

15 Intermediate model confusion matrices for Runs 1–3. 37

16 Intermediate model confusion matrices for Runs 4–6. 37

17 Intermediate model ROC curves for Runs 1–3. 38
18 Intermediate model ROC curves for Runs 4–6. 38
19 Final model confusion matrices for Runs 1–3. 38
20 Final model confusion matrices for Runs 4–6. 39
21 Final model ROC curves for Runs 1–3. 39
22 Final model ROC curves for Runs 4–6. 39
23 A simple representation of the Google Colab notebook structure for the FFT-based deepfake detector. 40

List of Tables

1 Baseline 3D CNN classification report (Run 5, pixel-domain video frames). 20
2 Intermediate FFT-based ResNet classification report (Run 3, frequency-domain images). 21
3 Final fine-tuned FFT-based ResNet classification report (Run 2, best-performing model). 22
4 Comparison of accuracy and latency characteristics across model stages. Latency was explicitly profiled only for the final model. 24
5 Summary of test accuracy and AUC across runs (mean over six runs). 25
6 Per-run test accuracy and AUC for baseline, intermediate, and final models. 30
7 Run 1: Final fine-tuned model classification report. 31
8 Run 2: Final fine-tuned model classification report (best AUC). . . . 32
9 Run 3: Final fine-tuned model classification report. 32
10 Run 4: Final fine-tuned model classification report. 33
11 Run 5: Final fine-tuned model classification report. 33

12 Run 6: Final fine-tuned model classification report. 34

1 Introduction

High-quality deepfake videos have moved from research novelty to everyday reality. Generative adversarial networks (GANs) and diffusion models [6, 9] can produce photorealistic face swaps and synthetic personas that are difficult for human viewers to distinguish from authentic footage. When such content is introduced as "evidence" in legal, investigative, or corporate settings, it undermines trust in digital media and complicates the task of establishing what actually happened in the world.

Most existing deepfake detectors work in the spatial domain [10, 15]. They operate directly on raw RGB frames and look for visual artifacts: inconsistent blinking patterns, boundary glitches around the face, lighting mismatches, or subtle warping artifacts. While these techniques can be effective against specific deepfake generators, they are locked into an arms race. As generative models get better, many of the pixel-level defects they exploit simply disappear, leaving detectors brittle and prone to sudden performance drops when a new model is released or a different compression pipeline is used.

In this capstone we explore an alternative path: analyzing deepfakes in the frequency domain. Instead of learning directly from pixels, we transform video frames using the two-dimensional Fast Fourier Transform (2D FFT) and train convolutional networks on log-magnitude spectra. The intuition is that the deepfake generation and encoding process may leave behind regularities in frequency space—for example, odd correlations between high-frequency bands or unnatural energy distributions—that are less visible but more stable across generators.

Our central research question is:

> *Can a frequency-domain deepfake detector provide higher accuracy and more stable performance than a purely pixel-domain baseline, while still supporting low-latency forensic triage?*

Low latency is critical in forensic workflows where investigators must quickly triage large volumes of video—for example, during time-sensitive investigations or when screening incoming digital evidence in a lab setting.

To answer this question we design a three-stage experimental pipeline:

1. A baseline 3D CNN that ingests short RGB video clips and learns directly in the pixel domain.

2. An intermediate frequency-domain model that converts frames to FFT magnitude spectra and trains a ResNet50 via transfer learning.

3. A final fine-tuned model that unfreezes the top convolutional block of ResNet50 and measures inference latency on an NVIDIA Tesla T4 GPU.

Compared with prior frequency-domain detectors such as FreqNet [17], which are primarily evaluated as generic cross-source deepfake classifiers, our work emphasizes an end-to-end pipeline for forensic triage. We explicitly combine FFT-based representations with transfer learning and fine-tuning, measure latency on realistic hardware, and position the final model as a practical tool for low-latency evidence screening rather than purely as a benchmark detector.

By running this pipeline six times with different random splits, we compare accuracy, AUC, and error patterns across models, and we measure per-frame runtime for the final detector. The remainder of this report details related work, describes our methodology, presents experimental results, and discusses implications for digital forensics.

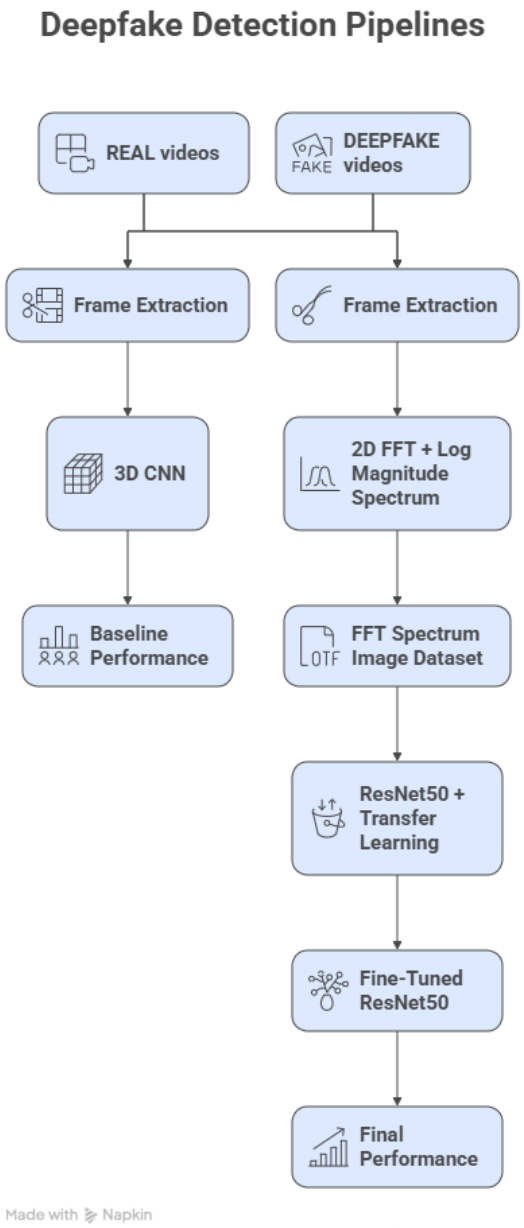


Figure 1: High-level deepfake detection pipeline showing the baseline pixel-domain 3D CNN, the FFT-based ResNet model, and the fine-tuned final model.

2 Background and Related Work

2.1 Deepfake Generation and Threats

The earliest face-swapping systems relied on classical computer-graphics pipelines and required extensive manual tuning. Modern deepfakes, by contrast, are typically generated using deep neural networks such as autoencoders, GANs, or diffusion models. These systems can learn mappings between source and target identities using large datasets of face images or videos, producing high-resolution outputs with realistic lighting, shading, and expressions.

From a security and forensic perspective, deepfakes create several problems. They can be used to fabricate incriminating videos, impersonate public figures, or generate non-consensual explicit content. Even when clearly fake, their existence fuels the "liar's dividend," where genuine footage can be dismissed as fabricated because deepfake technology is known to exist.

2.2 Pixel-Domain Detection

Many early detectors focus on hand-designed cues in the spatial domain. Examples include detectors that track inconsistent eye blinking, head pose dynamics, or facial landmark trajectories, as well as CNN-based methods that exploit face warping artifacts [10]; detectors that exploit color mismatches between face and background; and methods that learn directly from image patches using 2D or 3D CNNs. These systems typically treat deepfake detection as a supervised binary classification problem: given a set of real and fake frames or clips, train a network to output the correct label. Physiological approaches such as remote photoplethysmography (rPPG) extract pulse signals from facial videos and have been proposed as complementary cues for liveness and deepfake detection [14].

Pixel-domain methods can achieve high accuracy on benchmarks that closely

match their training data. However, they often struggle with cross-dataset generalization. When compression settings change, when videos are re-encoded by a social media platform, or when a new deepfake generator is introduced, spatial artifacts may be suppressed or appear in new forms. As a result, many pixel-based detectors degrade significantly in the wild.

2.3 Frequency-Domain Approaches

A growing line of research investigates the frequency domain as a more stable feature space. Several authors have shown that generative models can leave characteristic patterns in the power spectrum of images, especially at high frequencies or along specific directions. Multiple analyses have shown that modern generative models and common upsampling techniques leave systematic patterns in the frequency spectrum of generated images [5, 4]. These include peaks, periodic artifacts, and unnatural high-frequency energy distributions that are tied to architectural choices such as transposed convolutions and upsampling layers. More recent work like FreqNet explicitly learns in frequency space to improve cross-source generalization [17], while other studies caution that generators can be modified to conceal spectral fingerprints [3]. Some methods apply FFTs to full images and train CNNs on magnitude spectra; others analyze local frequency patches or use wavelet transforms.

Recent large-scale benchmarks such as the DeepFake Detection Challenge (DFDC) [2] and Celeb-DF [11] have pushed both scale and realism in deepfake forensics. DFDC, for example, provides hundreds of thousands of clips with diverse actors, environments, and compression settings, while Celeb-DF focuses on high-quality celebrity manipulations that are visually challenging even for humans. In this project we focus on FaceForensics++ [15] because it offers well-curated pairs of real and manipulated videos with clear labels and manageable computational requirements for a capstone-scale study, while still being representative enough to evaluate frequency-domain de-

tectors.

Frequency-domain approaches are appealing for at least two reasons. First, they explicitly capture global structure and periodic patterns that might be hard to learn from local convolutions alone. Second, certain image processing operations—including common lossy compression schemes—have predictable effects in frequency space, which may make it easier to reason about robustness.

Our work builds on this line of research by constructing a full end-to-end pipeline: we start with a pixel-domain 3D CNN baseline, then replace the input representation with FFT spectra and reuse a strong pre-trained architecture (ResNet50) via transfer learning, and finally fine-tune to optimize both accuracy and latency for a forensic triage setting.

3 Methodology

3.1 Data and Experimental Setup

We use the FaceForensics++ dataset, which provides pairs of real and manipulated videos produced by several deepfake generation methods [15]. For each run we select up to 200 real and 200 fake videos from the dataset, keeping only standard video formats (e.g., MP4, AVI, MOV, MKV, WEBM). To control runtime, we operate on a fixed number of frames per video rather than using full-length clips. Other large-scale benchmarks such as the DeepFake Detection Challenge (DFDC) [2] and Celeb-DF [11] have pushed scale and realism in recent work, but for this capstone we focus on FaceForensics++ as a well-understood reference dataset.

All experiments are conducted in Google Colab Pro on a virtual machine with a Tesla T4 GPU and high-memory runtime. Code is written in Python using PyTorch [12], OpenCV [1], NumPy, and scikit-learn [13]. Video and image assets are stored in Google Drive and accessed through Colab.

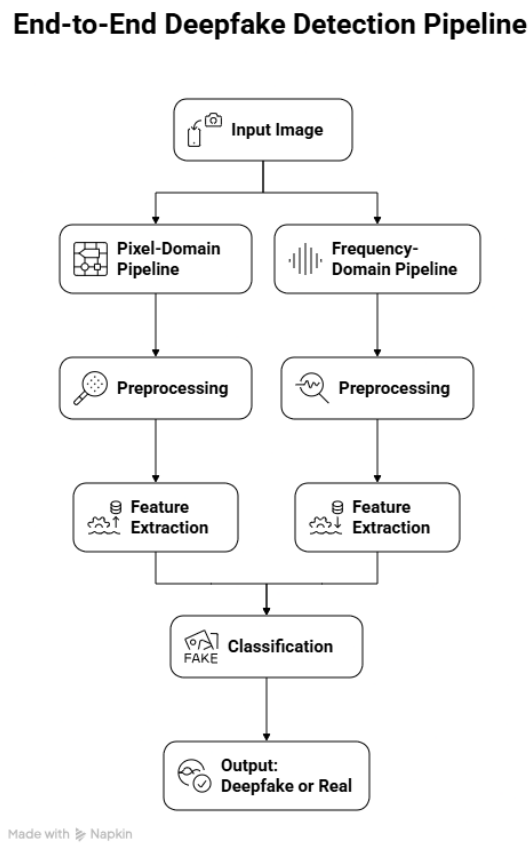


Figure 2: End-to-end deepfake detection pipeline: from raw videos to frame extraction, FFT conversion, model inference, and forensic reporting.

3.2 Data Loading and Frame Extraction

We implement a reusable video loader in `data_loader.py`. Given a path to a video, the function `extract_frames()` opens the file using OpenCV's `VideoCapture` interface and returns a list of RGB frames. For the baseline 3D CNN, we:

- Sample up to 16 frames per video using either sequential reading or evenly spaced indices when the total frame count is known.
- Resize each frame to 224×224 pixels.
- Convert from BGR (OpenCV default) to RGB and normalize pixel values to the $[0, 1]$ range.

A custom `VideoFrameDataset` wraps these steps and returns tensors in the shape (C, T, H, W), where $C = 3$ channels, $T = 16$ frames, and $H = W = 224$ pixels. We split the dataset into 50% training, 25% validation, and 25% test sets using PyTorch's `random_split`. A custom `collate_fn` silently skips any videos for which frame extraction fails.

3.3 Baseline Model: 3D CNN in the Pixel Domain

To establish a baseline, we train a relatively shallow 3D CNN that operates on RGB frame sequences:

- Three 3D convolutional layers with kernel size 3 and padding 1, each followed by ReLU activation and 3D max pooling.
- After the final pooling layer, we flatten to a fully connected layer of size 512 with dropout (0.5), followed by a 2-unit output layer for the `REAL` vs. `FAKE` classes.

We train this model for 30 epochs using Adam with a learning rate of 0.001 and cross-entropy loss. Each epoch reports training and validation loss and accuracy.

3.4 Frequency-Domain Preprocessing

The frequency-domain pipeline starts at the frame level. For each extracted RGB frame, we:

1. Convert the frame to grayscale.
2. Compute the 2D FFT and apply `fftshift` to center the low-frequency components.
3. Take the log-magnitude spectrum using `log1p` to compress the dynamic range.

4. Normalize the spectrum to the $[0, 255]$ range and save as an 8-bit PNG image.

We organize these images into a new directory structure with separate `real` and `fake` subfolders. Each image is named using the source video filename and frame index. Figure 3 illustrates this funnel from video frames to FFT images.

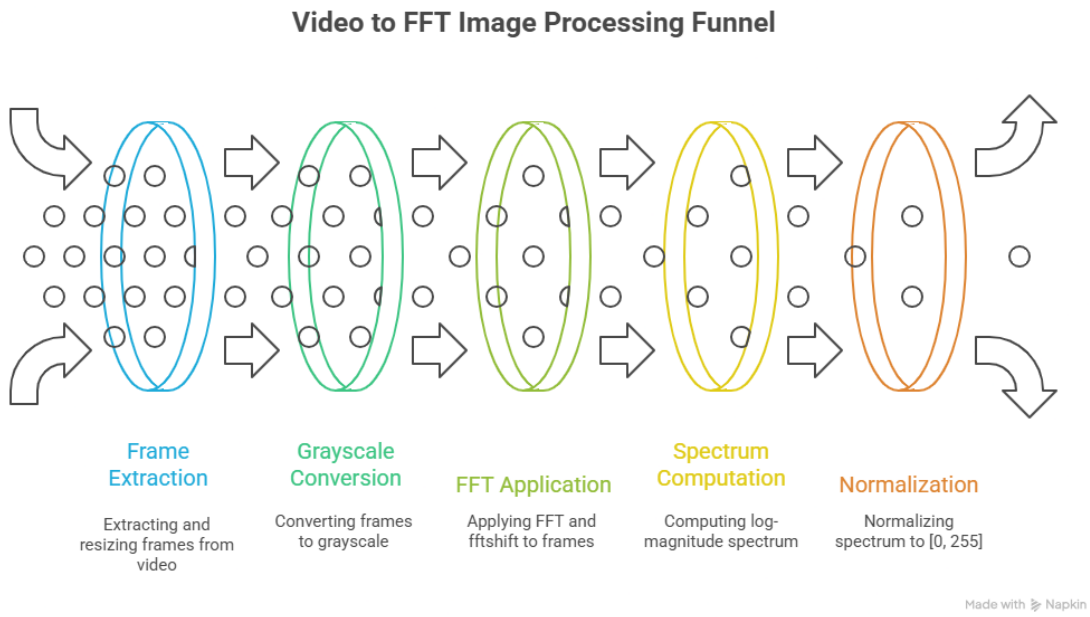


Figure 3: From video frames to frequency-domain representation: each RGB frame is converted to grayscale, transformed with a 2D FFT, shifted to center low frequencies, and mapped to a log-magnitude spectrum saved as an image.

3.5 Intermediate Model: ResNet50 with Transfer Learning

Our intermediate model treats each FFT spectrum as a standalone 2D image. We rely on the pre-trained ResNet50 architecture from `torchvision`:

- We load ResNet50 with ImageNet weights and freeze all layers except the final fully connected layer.
- The last layer is replaced with a new linear layer mapping the 2048 feature units to 2 output classes.

- We apply the recommended ImageNet preprocessing transforms (resize, center crop to 224×224, normalization with ImageNet mean and standard deviation).

A custom `FFTImageDataset` reads PNG images from the `real` and `fake` directories, converts grayscale images to 3-channel RGB, and applies the ResNet50 transforms. As before, we use a 50/25/25 split for training, validation, and testing. Using a batch size of 16, we train only the new final layer for 30 epochs with Adam and a learning rate of 0.001.

3.6 Final Model: Fine-Tuning ResNet50

To push performance further, we fine-tune the top convolutional block (`layer4`) of ResNet50 along with the new classification head:

- We unfreeze all parameters in `layer4` and the final fully connected layer.
- We instantiate a new optimizer that updates only parameters with `requires_grad = True`, using Adam with a very small learning rate of 1×10^{-5}.
- We run an additional 30 epochs of training using the same train/validation loaders and loss function.

After fine-tuning, we evaluate on the held-out test set and save the model state as `FINAL_finetuned_model_new.pth`. We repeat the entire process—baseline, FFT conversion, intermediate training, fine-tuning, and evaluation—six times with different random splits to assess stability.

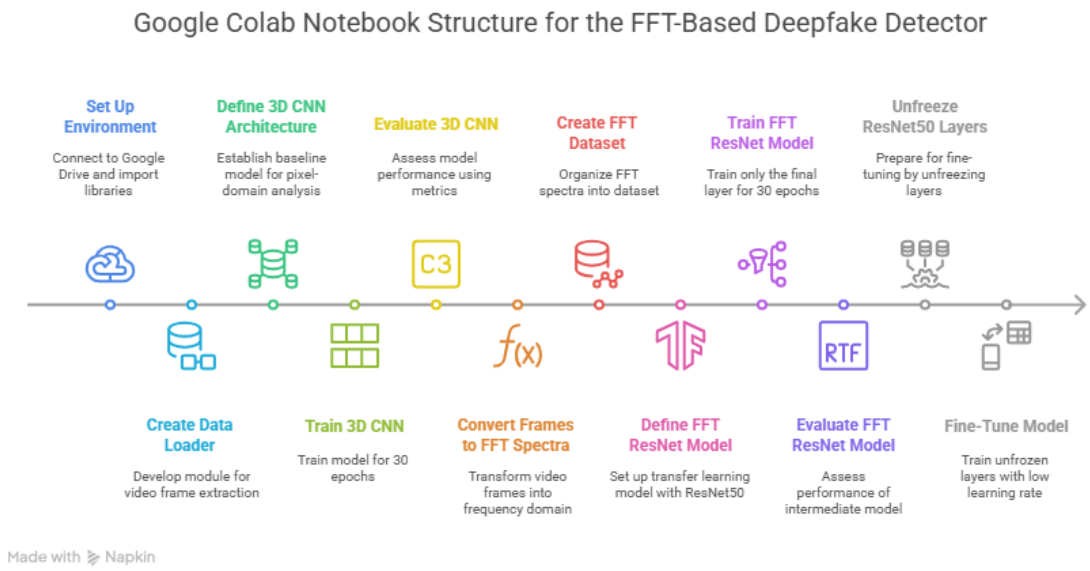


Figure 4: Google Colab notebook structure for the FFT-based deepfake detector, showing data loading, baseline training, FFT preprocessing, transfer learning, fine-tuning, evaluation, and latency measurement cells.

3.7 Implementation in Google Colab

All experiments were implemented in a single Google Colab notebook backed by Google Drive storage and a Tesla T4 GPU runtime. The notebook is organized into two logical paths: a pixel-domain baseline and a frequency-domain experimental pipeline.

Cells 1–3 perform environment setup: mounting Google Drive, creating the project root directory, and writing a reusable `data_loader.py` helper that exposes an `extract_frames()` function for reading video clips. Cells 4–7 implement the baseline 3D CNN path: they construct the `VideoFrameDataset` and 3D dataloaders, define the `Simple3DCNN` architecture, train it for 30 epochs, and then compute classification reports, confusion matrices, and ROC curves for the held-out test set.

Cells 8–17 implement the frequency-domain path. Cells 8–9 convert RGB frames to log-magnitude FFT spectra and save them as PNG images in `frequency_dataset/real`

and `frequency_dataset/fake`. Cells 10–13 define the `FFTImageDataset`, load the spectra with ImageNet-compatible transforms, and train a ResNet50 via transfer learning on these images. Cells 14–15 unfreeze the top convolutional block (`layer4`) of ResNet50 and fine-tune the network with a small learning rate, while Cells 16–17 perform final evaluation and save the fine-tuned weights as `FINAL_finetuned_model_new.pth`. Additional cells added later measure inference latency for the final model on the Tesla T4 GPU.

A more detailed, cell-by-cell guide is provided in Appendix D.

3.8 Evaluation Metrics

Across all stages we compute:

- **Accuracy**: fraction of correct predictions.
- **Precision, recall, F1-score**: reported per class (`REAL`, `FAKE`) and as macro- and weighted averages.
- **Confusion matrices**: to visualize error patterns.
- **ROC curves and AUC**: using the probability of the `FAKE` class as the positive score.

We also measure inference latency for the final model by timing batched forward passes on the Tesla T4, computing median and 95th percentile per-batch and per-image times.

4 Results

4.1 Baseline 3D CNN Performance

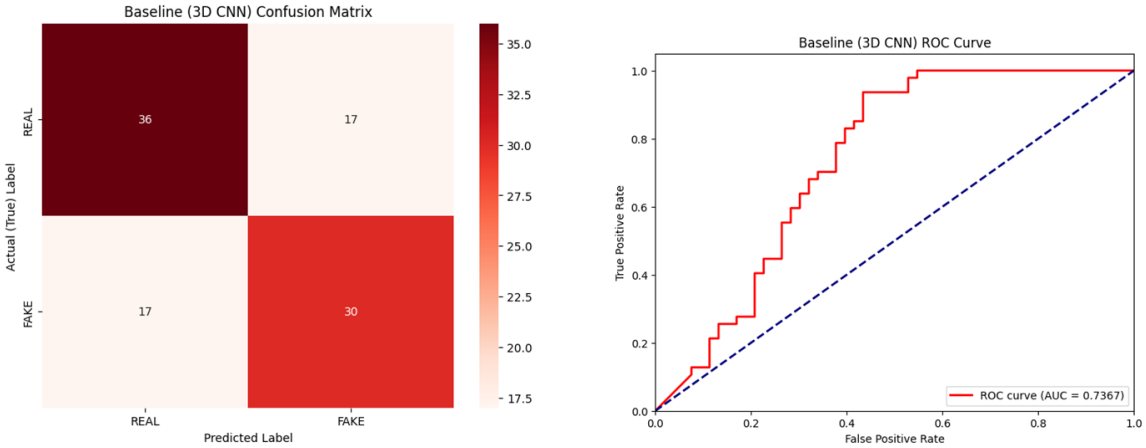
Table 1: Baseline 3D CNN classification report (Run 5, pixel-domain video frames).

Class	Precision	Recall	F1-score	Support
REAL	0.68	0.68	0.68	53
FAKE	0.64	0.64	0.64	47
Accuracy	N/A	N/A	0.66	100
Macro avg	0.66	0.66	0.66	100
Weighted avg	0.66	0.66	0.66	100

Table 1 summarizes the baseline test accuracy and AUC across six runs of the 3D CNN. Each run uses the same architecture but different random splits.

Overall, baseline performance is modest. Accuracy hovers around 57–60% for most runs, and AUC values in the 0.55–0.65 range indicate that the model only modestly separates real from fake videos. The best run (Run 5) reaches 66% accuracy and an AUC of 0.7367, but still falls short of what we would want from a forensic triage tool.

Confusion matrices show a mix of false positives and false negatives, with a slight tendency to misclassify fake videos as real in some runs and the reverse in others. The ROC curves are only slightly above the diagonal, which is consistent with the AUC scores.



(a) Baseline 3D CNN confusion matrix.

(b) Baseline 3D CNN ROC curve.

Figure 5: Baseline pixel-domain model performance on the test set.

4.2 Intermediate Frequency-Domain Model

Table 2: Intermediate FFT-based ResNet classification report (Run 3, frequency-domain images).

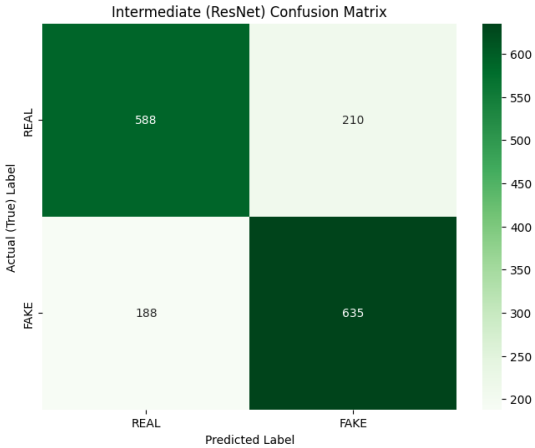
Class	Precision	Recall	F1-score	Support
REAL	0.76	0.74	0.75	798
FAKE	0.75	0.77	0.76	823
Accuracy	N/A	N/A	0.75	1621
Macro avg	0.75	0.75	0.75	1621
Weighted avg	0.75	0.75	0.75	1621

Table 2 reports accuracy and AUC for the intermediate ResNet50 model trained on FFT magnitude spectra.

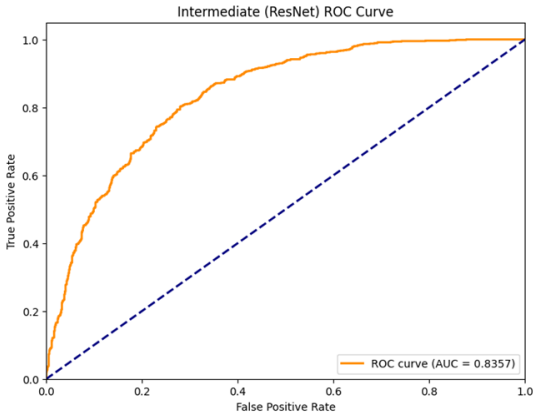
Switching to the frequency domain and leveraging a pre-trained ResNet50 immediately boosts performance. Accuracy improves by roughly 15–20 percentage points over the baseline, and AUC values in the 0.82–0.85 range indicate much stronger class separability. Qualitatively, the confusion matrices show more mass along the

diagonal: many more real and fake videos are correctly classified.

These results suggest that the FFT representation captures features that are both discriminative and easier to learn using a deep architecture pre-trained on large-scale natural images, even though those images are not deepfakes.



(a) Intermediate FFT-based ResNet confusion matrix.



(b) Intermediate FFT-based ResNet ROC curve.

Figure 6: Intermediate frequency-domain model performance on the FFT image test set.

4.3 Final Fine-Tuned Model

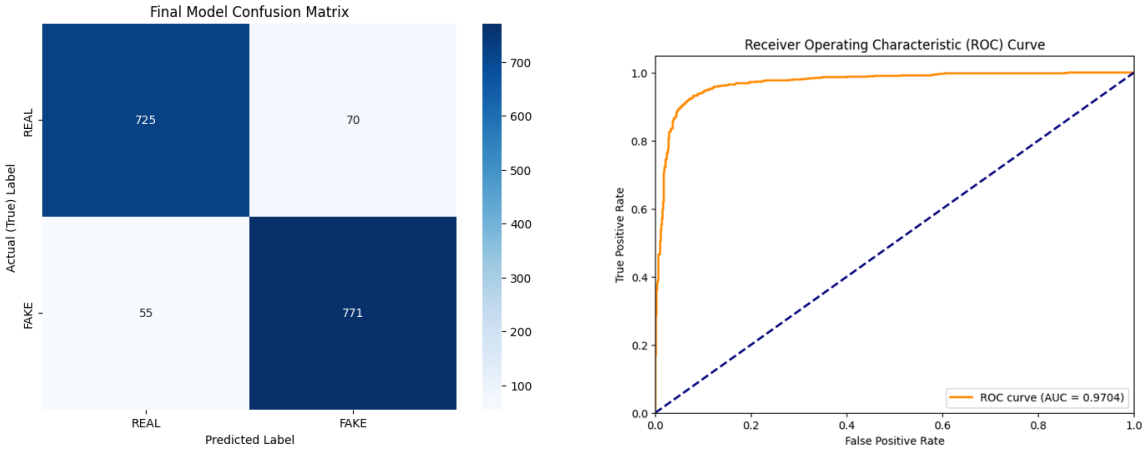
Table 3: Final fine-tuned FFT-based ResNet classification report (Run 2, best-performing model).

Class	Precision	Recall	F1-score	Support
REAL	0.93	0.91	0.92	795
FAKE	0.92	0.93	0.93	826
Accuracy	N/A	N/A	0.92	1621
Macro avg	0.92	0.92	0.92	1621
Weighted avg	0.92	0.92	0.92	1621

Fine-tuning the top convolutional block of ResNet50 leads to our best performing model. Table 3 summarizes the results.

Across all six runs, test accuracy remains between 90% and 92%, and AUC values are consistently above 0.95, peaking at 0.9704 in Run 2. This represents a substantial improvement over both the baseline and intermediate models. In other words, fine-tuning adapts the top-level filters of ResNet50 to the particular structure of FFT spectra derived from deepfakes.

Confusion matrices for the final model show a strong diagonal pattern. For Run 2, for example, 725 of 795 real images and 771 of 826 fake images are classified correctly. ROC curves closely hug the top-left corner of the plot, indicating that the model maintains high true positive rates even at low false positive rates.



(a) Final fine-tuned model confusion matrix (Run 2).

(b) Final fine-tuned model ROC curve (Run 2, AUC = 0.9704).

Figure 7: Final frequency-domain model performance on the test set.

4.4 Latency Measurements

Because our goal is a low-latency forensic triage tool, we measured inference speed of the final model on the Tesla T4 GPU using a batch size of 16 FFT images. We ran multiple forward passes, discarded warm-up iterations, and computed the median and 95th percentile over batch times and per-image times.

For the configuration used in Run 2, we observed:

- Median batch time: 43.48 ms
- 95th percentile batch time: 44.57 ms
- Median per-image time: 2.72 ms
- 95th percentile per-image time: 2.79 ms

Even accounting for pre- and post-processing overhead, these numbers are encouraging: Table 4 summarizes accuracy and latency characteristics across all three stages and shows that the final model processes frames in roughly 2–3 ms each on a Tesla T4. In contrast, the baseline 3D CNN operates on short video clips with 3D convolutions and has a substantially higher computational cost per decision, making it less attractive for large-scale triage. The FFT-based ResNet therefore achieves a better balance between detection quality and practical runtime.

Table 4: Comparison of accuracy and latency characteristics across model stages. Latency was explicitly profiled only for the final model.

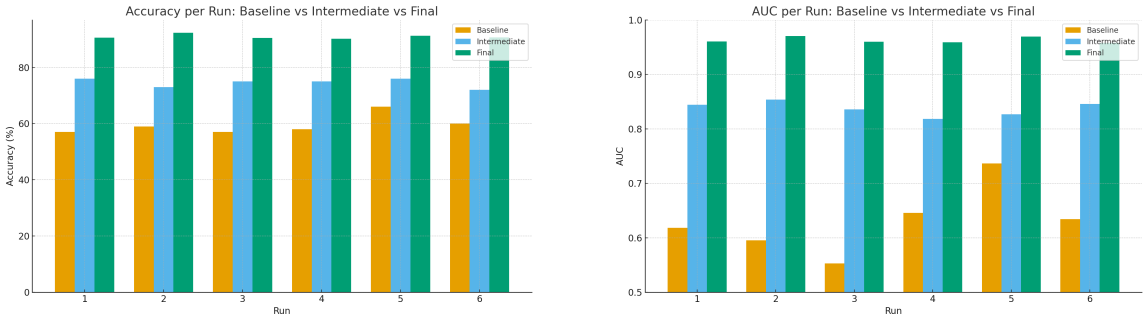
Model stage	Test accuracy (range, %)	Latency summary
Baseline 3D CNN	57–66	Clip-level 3D CNN; higher, not profiled
Intermediate FFT ResNet	72–76	Frame-level 2D CNN; moderate, not profiled
Final fine-tuned FFT ResNet	90–92	Frame-level; 43.48 ms/batch (16), 2.72 ms/frame

4.5 Summary of Improvements Across Stages

To highlight the progression from baseline to final model, we plotted accuracy and AUC across runs for all three stages (figures to be inserted). The key trends are:

- Accuracy: baseline (57–66%), intermediate (72–76%), final (90–92%).
- AUC: baseline (0.55–0.74), intermediate (0.82–0.85), final (0.96–0.97).

The frequency-domain representation plus transfer learning yields a large jump from the baseline, and fine-tuning delivers another significant gain without sacrificing latency.



(a) Accuracy per run for baseline, FFT ResNet, and final models.

(b) AUC per run for baseline, FFT ResNet, and final models.

Figure 8: Performance comparison across the three stages of the pipeline.

4.6 Cross-Run Accuracy and AUC Statistics

Table 5 summarizes test accuracy and AUC averaged across the six independent runs for each model stage. The baseline 3D CNN achieves roughly 59.5% accuracy with an average AUC of 0.63, the intermediate FFT-based ResNet improves this to 74.5% accuracy and an AUC of 0.84, and the final fine-tuned model reaches about 90.9% accuracy with an average AUC of 0.96.

Table 5: Summary of test accuracy and AUC across runs (mean over six runs).

Model stage	Mean test accuracy (%)	Mean AUC
Baseline 3D CNN	59.5	0.63
Intermediate FFT ResNet	74.5	0.84
Final fine-tuned FFT ResNet	90.9	0.96

5 Discussion

5.1 Why the Frequency Domain Helps

The improvement from the 3D CNN to the FFT-based ResNet50 suggests that frequency-domain cues are more informative and stable than the pixel-level artifacts captured by the baseline. One plausible explanation is that the generative process introduces subtle but systematic patterns in the spectrum of facial regions—for example, unnatural correlations between mid- and high-frequency components or regular banding effects.

These patterns may be difficult to express in a shallow 3D CNN trained from scratch on a relatively small dataset, but they align well with the kinds of filters learned by ResNet50 on large-scale natural images. By feeding log-magnitude spectra into a deep pre-trained model, we effectively repurpose its ability to detect textures, edges, and global patterns—just now in frequency space instead of the spatial domain.

The model-compression literature provides concrete tools for making deep detectors more practical in resource-constrained settings. Deep Compression combines pruning, quantization, and coding to significantly reduce parameter counts with minimal accuracy loss [7], while knowledge distillation transfers performance from a large teacher to a smaller student network [8]. Architectures such as MobileNetV2 are specifically designed for low-FLOP inference on mobile and edge devices [16].

5.2 Overfitting and Generalization

The baseline 3D CNN displays clear signs of overfitting. Training accuracy climbs toward 100% on some runs, while validation accuracy stagnates around 65–75%. This is not surprising: the model has many parameters relative to the amount of training data, and the 3D convolutions must simultaneously learn spatial and temporal features from scratch.

In contrast, the FFT-based ResNet50 starts from a strong prior and only trains a small number of parameters in the final layer during the intermediate stage. This keeps overfitting in check and supports better generalization, even before fine-tuning. When we unfreeze `layer4` for the final model, we deliberately use a very small learning rate to avoid catastrophic forgetting while still allowing the model to specialize.

5.3 Forensic Utility

From a forensic perspective, two aspects of our results are important:

- **High separability.** AUC values above 0.95 mean that the model can provide well-calibrated confidence scores. Investigators can choose thresholds that trade off false positives against false negatives depending on context.
- **Low latency.** Per-frame inference times around 2–3 ms on a mid-range GPU make it feasible to scan large volumes of video or support interactive review in a lab environment.

Our pipeline is not intended to replace full forensic analysis or courtroom expert testimony. Instead, it is designed as a triage tool: something that can quickly flag suspicious clips, prioritize human review, and provide structured output (confusion matrices, ROC curves, and log files) that can be incorporated into broader investigative workflows.

6 Limitations and Future Work

This project has several limitations that should be kept in mind when interpreting the results:

- **Dataset representativeness.** We rely on FaceForensics++, a widely used benchmark but still a curated dataset. Real-world deepfakes may be gener-

ated by proprietary models, heavily compressed by social media platforms, or combined with other manipulations.

- **Frame-level analysis.** Our FFT pipeline treats frames independently. We do not model temporal consistency or exploit motion cues, which may carry additional forensic signals.
- **Limited architecture search.** We focused on ResNet50 for transfer learning. Other backbones or hybrid architectures (e.g., combining spatial and frequency inputs) might yield further gains.
- **Hardware assumptions.** Latency measurements are specific to a Tesla T4 GPU. Performance will differ on CPUs, mobile devices, or other accelerators.

Future work could address these limitations by:

- Evaluating the model on newer and more diverse deepfake datasets and real-world videos collected from social platforms.
- Incorporating temporal models such as 3D CNNs or transformers on top of frequency-domain features.
- Exploring lightweight architectures and pruning or quantization techniques to deploy the detector on edge devices.
- Extending the pipeline into a complete forensic tool with a graphical interface, case management, and chain-of-custody logging.

7 Conclusion

This capstone project set out to evaluate whether a frequency-domain approach could provide a more robust and efficient deepfake detector than a baseline pixel-domain

model. By implementing a three-stage pipeline—a 3D CNN baseline, an FFT-based ResNet50 intermediate model, and a fine-tuned final model—we demonstrated substantial improvements in both accuracy and AUC, while keeping inference latency low enough for practical forensic triage.

Our results show that operating in the frequency domain, combined with transfer learning and targeted fine-tuning, yields a detector that reliably distinguishes real from fake videos on FaceForensics++ with over 90% accuracy and AUC values above 0.95. Although more work is needed to validate this approach on real-world data and diverse generators, the findings support the broader idea that *how* we represent visual evidence—in this case, as spectra rather than pixels—can make a significant difference for digital forensics.

A Additional Results and Per-Run Metrics

This appendix summarizes per-run performance across the three stages of the pipeline: the baseline pixel-domain 3D CNN, the intermediate FFT-based ResNet50 with a frozen backbone, and the final fine-tuned FFT-based ResNet50 model.

Table 6 reports test accuracy and AUC for each of the six independent runs and each model stage. These values are consistent with the main text but collected here for ease of comparison and reference.

Table 6: Per-run test accuracy and AUC for baseline, intermediate, and final models.

Model stage	Run	Test accuracy (%)	AUC
Baseline 3D CNN	1	57.00	0.6184
	2	59.00	0.5954
	3	57.00	0.5530
	4	58.00	0.6455
	5	66.00	0.7367
	6	60.00	0.6341
Intermediate FFT ResNet	1	76.00	0.8441
	2	73.00	0.8537
	3	75.00	0.8357
	4	75.00	0.8181
	5	76.00	0.8268
	6	72.00	0.8459
Final fine-tuned FFT ResNet	1	90.56	0.9604
	2	92.29	0.9704
	3	90.50	0.9597
	4	90.25	0.9587
	5	91.24	0.9693
	6	90.68	0.9577

Hyperparameters are largely consistent across runs. For completeness, the key settings are:

- **Baseline 3D CNN:** 30 epochs; Adam optimizer (lr = 0.001); batch size = 4; 16 frames per video; input resolution 224×224.
- **Intermediate FFT ResNet:** 30 epochs; Adam on the final fully connected layer only (lr = 0.001); batch size = 16; ResNet50 initialized with ImageNet

weights.

- **Final fine-tuned FFT ResNet:** Additional 30 epochs; Adam optimizer on `layer4` + final layer (lr = 1×10^{-5}); batch size = 16; same FFT image representation as the intermediate stage.

B Complete Classification Reports

This appendix collects the full test-set classification reports for each of the six runs of the final fine-tuned FFT-based ResNet model. Each report shows precision, recall, F1-score, and support for the `REAL` and `FAKE` classes, along with macro and weighted averages.

Run 1 – Final Fine-Tuned Model

Table 7: Run 1: Final fine-tuned model classification report.

Class	Precision	Recall	F1-score	Support
REAL	0.93	0.87	0.90	803
FAKE	0.88	0.94	0.91	818
Accuracy			0.91	1621
Macro avg	0.91	0.91	0.91	1621
Weighted avg	0.91	0.91	0.91	1621

Run 2 – Final Fine-Tuned Model

Table 8: Run 2: Final fine-tuned model classification report (best AUC).

Class	Precision	Recall	F1-score	Support
REAL	0.93	0.91	0.92	795
FAKE	0.92	0.93	0.93	826
Accuracy			0.92	1621
Macro avg	0.92	0.92	0.92	1621
Weighted avg	0.92	0.92	0.92	1621

Run 3 – Final Fine-Tuned Model

Table 9: Run 3: Final fine-tuned model classification report.

Class	Precision	Recall	F1-score	Support
REAL	0.90	0.90	0.90	798
FAKE	0.91	0.91	0.91	823
Accuracy			0.90	1621
Macro avg	0.90	0.90	0.90	1621
Weighted avg	0.91	0.90	0.90	1621

Run 4 – Final Fine-Tuned Model

Table 10: Run 4: Final fine-tuned model classification report.

Class	Precision	Recall	F1-score	Support
REAL	0.92	0.88	0.90	805
FAKE	0.89	0.93	0.91	816
Accuracy			0.90	1621
Macro avg	0.90	0.90	0.90	1621
Weighted avg	0.90	0.90	0.90	1621

Run 5 – Final Fine-Tuned Model

Table 11: Run 5: Final fine-tuned model classification report.

Class	Precision	Recall	F1-score	Support
REAL	0.93	0.88	0.91	798
FAKE	0.89	0.94	0.92	823
Accuracy			0.91	1621
Macro avg	0.91	0.91	0.91	1621
Weighted avg	0.91	0.91	0.91	1621

Run 6 – Final Fine-Tuned Model

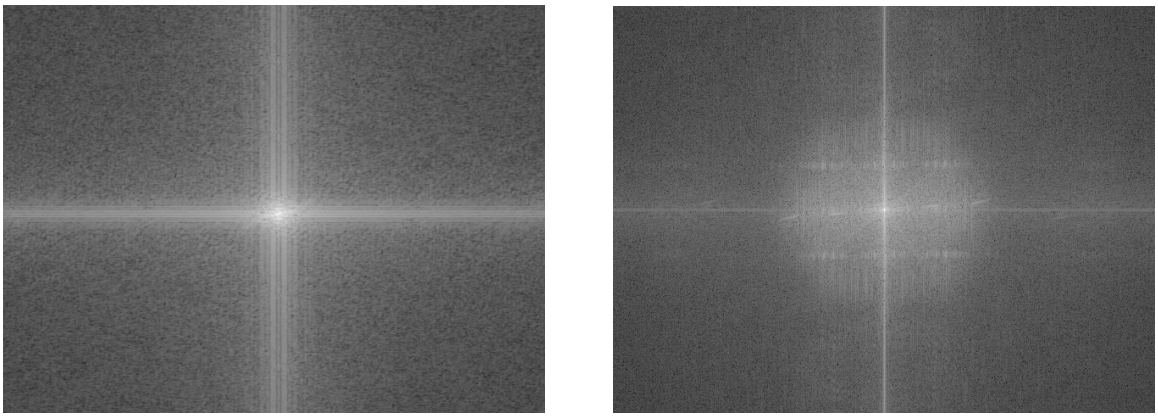
Table 12: Run 6: Final fine-tuned model classification report.

Class	Precision	Recall	F1-score	Support
REAL	0.91	0.90	0.90	796
FAKE	0.90	0.92	0.91	825
Accuracy			0.91	1621
Macro avg	0.91	0.91	0.91	1621
Weighted avg	0.91	0.91	0.91	1621

C Additional Figures

This appendix includes additional visualizations that complement the main results but are omitted from the body of the report to avoid clutter.

C.1 Example FFT Spectra and Training Curves



(a) Real frame FFT spectrum. (b) Fake frame FFT spectrum.

Figure 9: Example FFT magnitude spectra for representative real and fake frames, illustrating characteristic frequency-domain patterns.

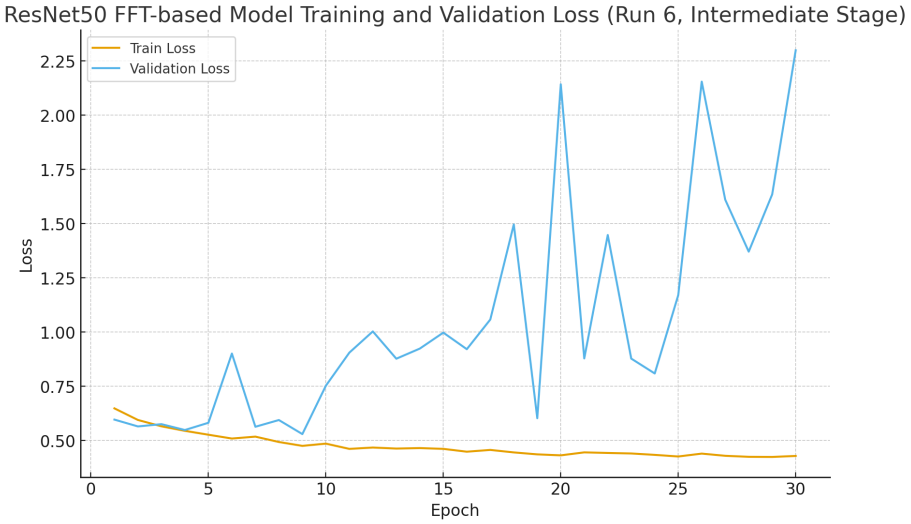
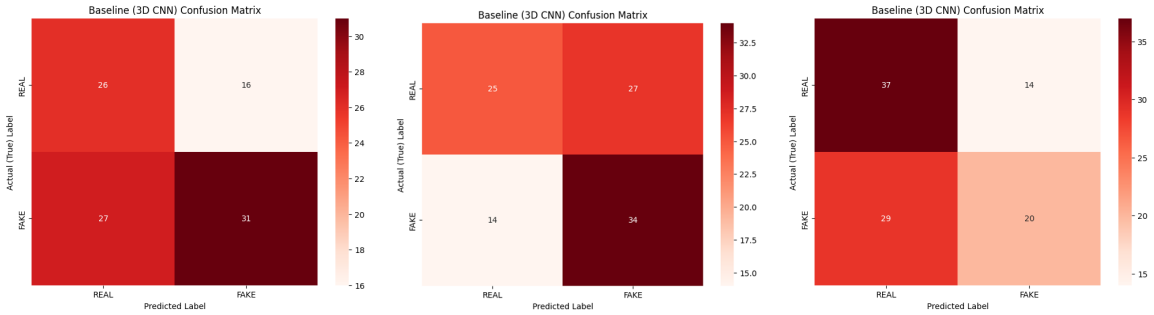


Figure 10: Training and validation loss curves for the FFT-based ResNet50 model (Run 6 intermediate stage) across 30 epochs.

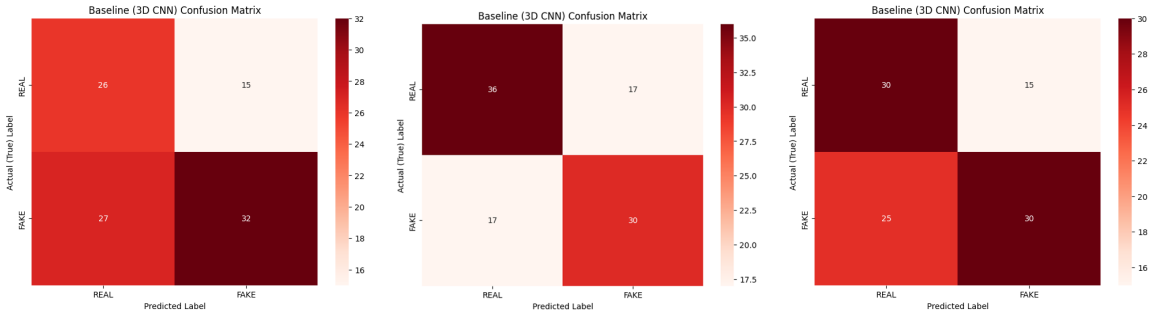
C.2 Baseline Model Confusion Matrices by Run

Figures 11 and 12 show the per-run confusion matrices for the baseline 3D CNN across all six runs.



(a) Run 1 (b) Run 2 (c) Run 3

Figure 11: Baseline model confusion matrices for Runs 1–3.

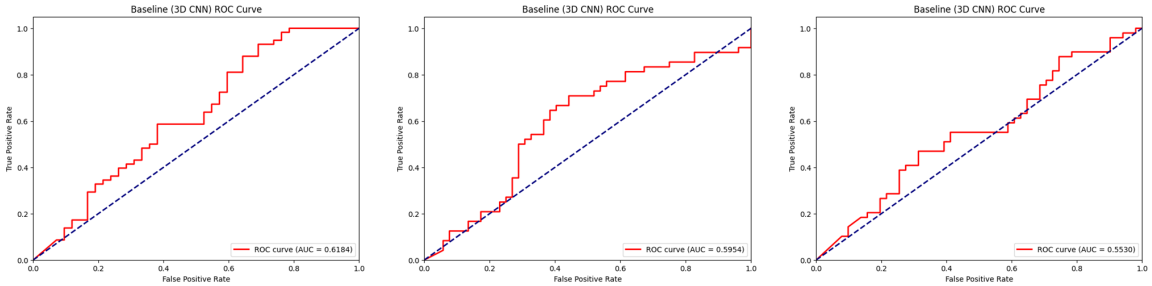


(a) Run 4 (b) Run 5 (c) Run 6

Figure 12: Baseline model confusion matrices for Runs 4–6.

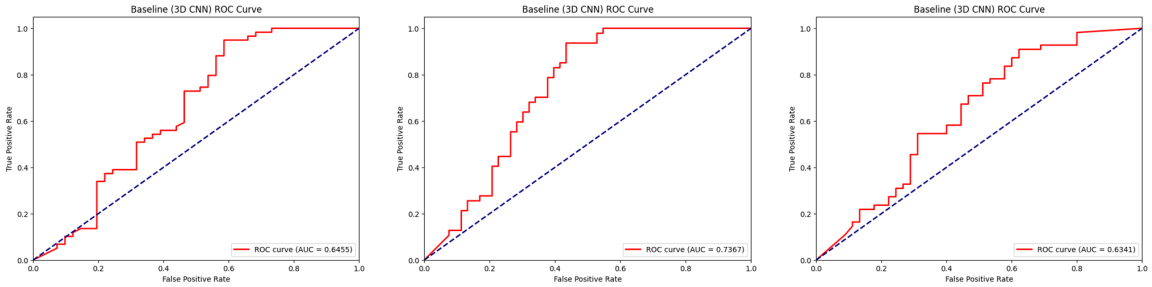
C.3 Baseline Model ROC Curves by Run

Figures 13 and 14 show the per-run ROC curves for the baseline 3D CNN.



(a) Run 1 (b) Run 2 (c) Run 3

Figure 13: Baseline model ROC curves for Runs 1–3.

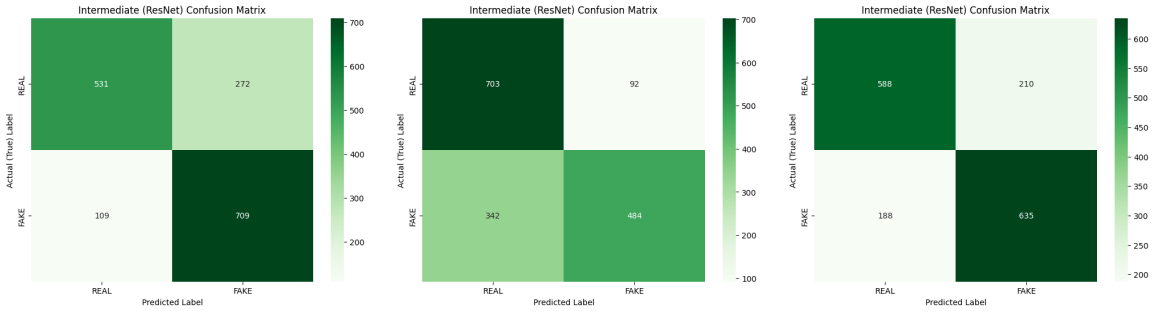


(a) Run 4 (b) Run 5 (c) Run 6

Figure 14: Baseline model ROC curves for Runs 4–6.

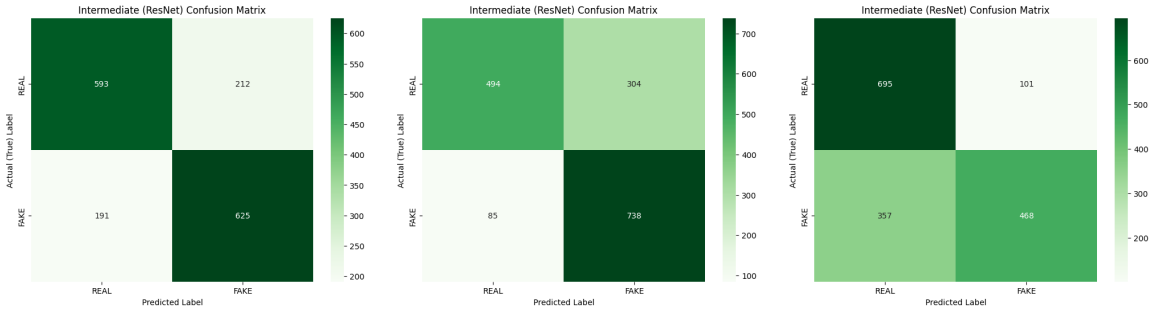
C.4 Intermediate Model Confusion Matrices by Run

Figures 15 and 16 show the per-run confusion matrices for the intermediate FFT-based ResNet50 model across all six runs.



(a) Run 1 (b) Run 2 (c) Run 3

Figure 15: Intermediate model confusion matrices for Runs 1–3.

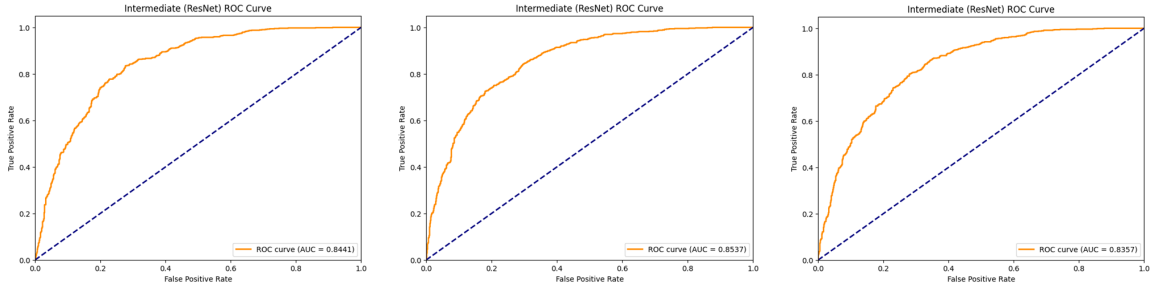


(a) Run 4 (b) Run 5 (c) Run 6

Figure 16: Intermediate model confusion matrices for Runs 4–6.

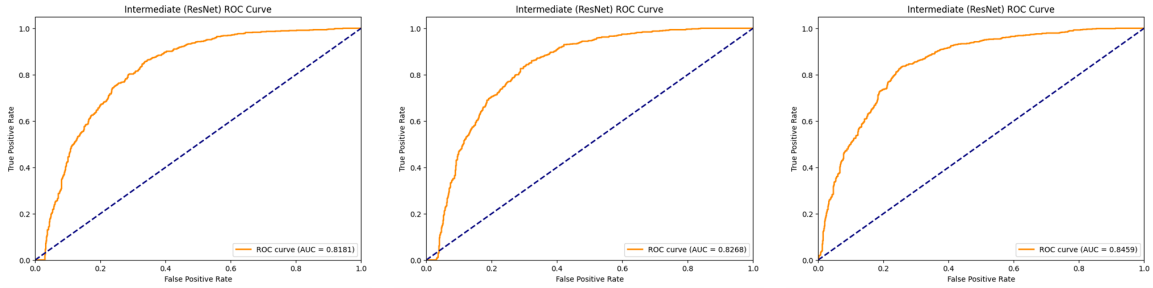
C.5 Intermediate Model ROC Curves by Run

Figures 17 and 18 show the per-run ROC curves for the intermediate FFT-based ResNet50 model.



(a) Run 1 (b) Run 2 (c) Run 3

Figure 17: Intermediate model ROC curves for Runs 1–3.

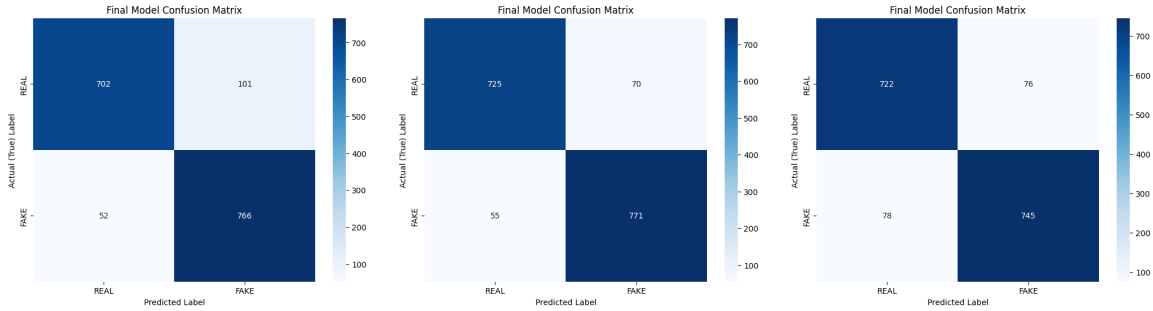


(a) Run 4 (b) Run 5 (c) Run 6

Figure 18: Intermediate model ROC curves for Runs 4–6.

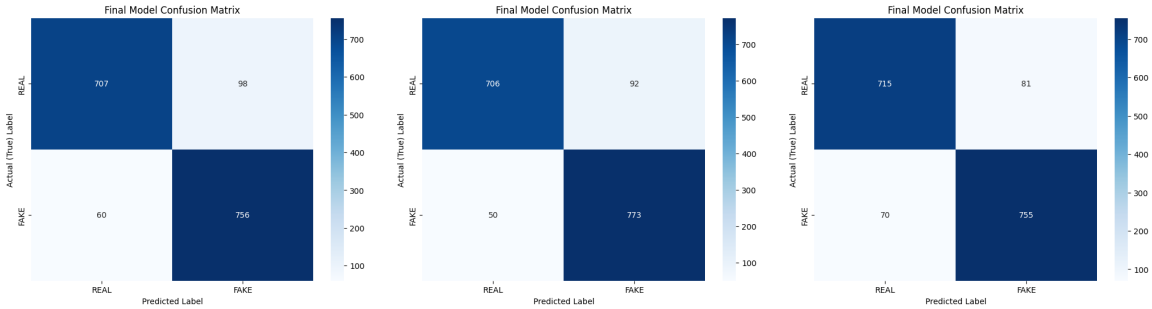
C.6 Final Model Confusion Matrices by Run

Figures 19 and 20 show the per-run confusion matrices for the final fine-tuned frequency-domain model across all six runs.



(a) Run 1 (b) Run 2 (c) Run 3

Figure 19: Final model confusion matrices for Runs 1–3.

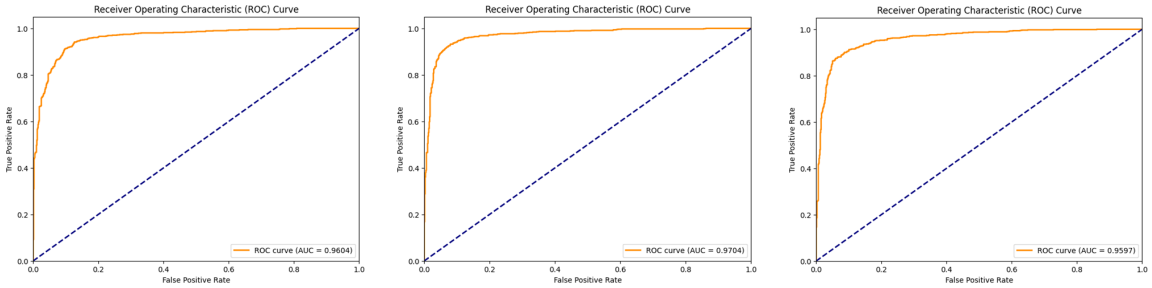


(a) Run 4 (b) Run 5 (c) Run 6

Figure 20: Final model confusion matrices for Runs 4–6.

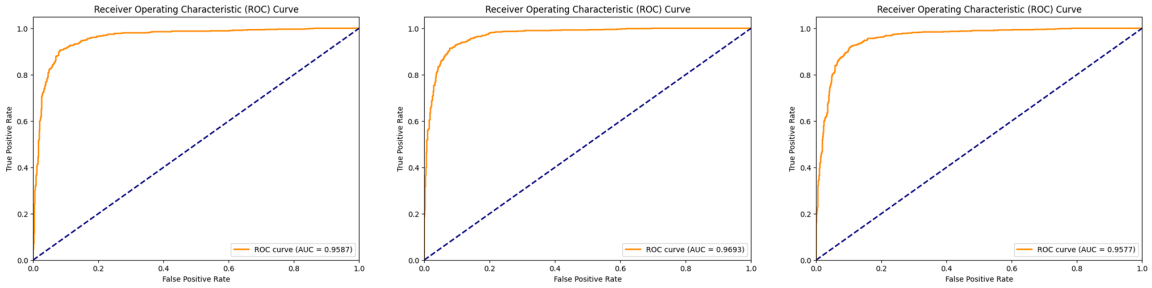
C.7 Final Model ROC Curves by Run

Figures 21 and 22 show the per-run ROC curves for the final fine-tuned model. These plots highlight the consistently high AUC values (> 0.95) across all runs.



(a) Run 1 (b) Run 2 (c) Run 3

Figure 21: Final model ROC curves for Runs 1–3.



(a) Run 4 (b) Run 5 (c) Run 6

Figure 22: Final model ROC curves for Runs 4–6.

D Google Colab Notebook Structure

This appendix summarizes the main cells in the Google Colab notebook used to run all experiments. The notebook is organized into two paths: a pixel-domain baseline (Cells 4–7) and a frequency-domain experimental pipeline (Cells 8–20).

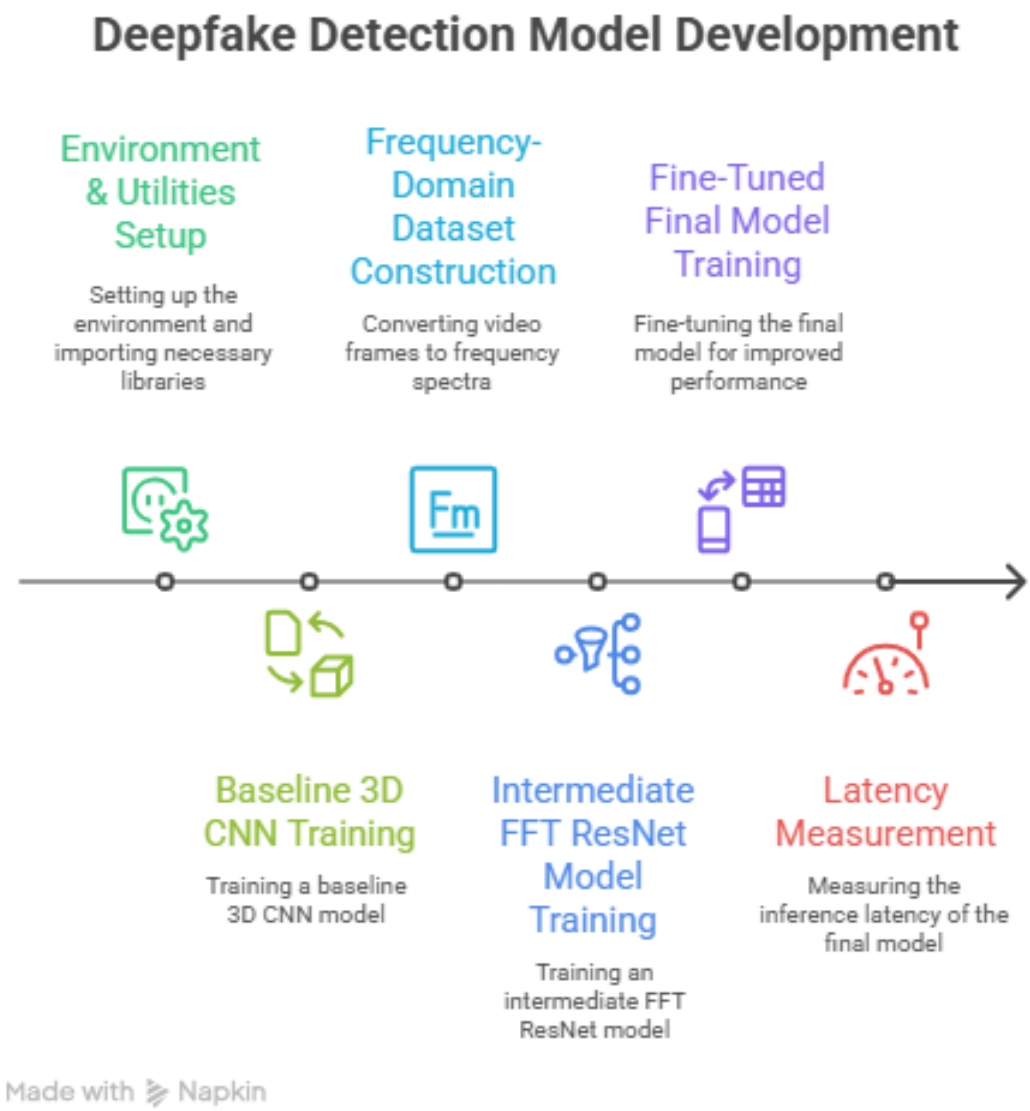


Figure 23: A simple representation of the Google Colab notebook structure for the FFT-based deepfake detector.

Part 1: Setup (Cells 1–3)

Cell 1: Environment setup. Mounts Google Drive, defines the `capstone_path` project root, and updates `sys.path` so that custom helper scripts can be imported.

Cell 2: Helper script `data_loader.py`. Writes a Python file containing the `extract_frames()` function, which opens a video using OpenCV and returns a list of RGB frames. This function is reused in both the baseline and frequency-domain pipelines.

Cell 3: Imports and device setup. Imports PyTorch, torchvision, NumPy, OpenCV, and other utilities; imports `extract_frames` from `data_loader.py`; and selects the computation device (`cuda` if available).

Path 1: Baseline Pixel-Domain Model (Cells 4–7)

Cell 4: Baseline data pipeline. Defines `VideoFrameDataset`, scans the `Real_videos` and `Deepfakes_videos` directories, and builds 50/25/25 train/validation/ test splits with corresponding PyTorch `DataLoader`s for the 3D CNN.

Cell 5: Baseline 3D CNN architecture. Implements `Simple3DCNN`, a shallow 3D convolutional network with three 3D conv–pool blocks followed by a fully connected layer and dropout, producing two logits for the REAL/FAKE classes.

Cell 6: Baseline model training. Defines `train_one_epoch` and `validate_one_epoch`, then trains `Simple3DCNN` for 30 epochs using Adam and cross-entropy loss, logging training and validation loss/accuracy per epoch.

Cell 7: Baseline evaluation. Runs the trained 3D CNN on the test set, collects predictions and class probabilities, and generates classification reports, confusion matrices, and ROC/AUC curves for the baseline.

Path 2: Frequency-Domain Model (Cells 8–20)

Cell 8: FFT conversion function. Defines `convert_frame_to_fft_spectrum()`, which converts an RGB frame to grayscale, applies a 2D FFT and frequency shift, and outputs a log-magnitude spectrum. A sample frame is visualized to verify the transformation.

Cell 9: FFT dataset generation. Loops over all videos, extracts up to 16 frames per clip, applies the FFT conversion, normalizes spectra into $[0, 255]$, and saves them as PNG images in `frequency_dataset/real` and `frequency_dataset/fake`.

Cell 10: FFT image dataset and transforms. Defines `FFTImageDataset` for loading the saved FFT PNGs, converts them to 3-channel images, and applies the recommended ResNet50 transforms (resize, crop, and ImageNet normalization). Verifies tensor shapes with a small test batch.

Cell 11: ResNet50 transfer-learning model. Loads a pre-trained ResNet50 with ImageNet weights, freezes all convolutional layers by default, and replaces the final fully connected layer with a 2-class (REAL/FAKE) classifier.

Cell 12: Intermediate model training. Trains only the new final layer of ResNet50 for 30 epochs on the FFT dataset using Adam with a learning rate of 0.001, reusing the shared `train_one_epoch` and `validate_one_epoch` routines.

Cell 13: Intermediate evaluation. Evaluates the frozen ResNet50+FFT model on the FFT test set, producing classification reports, confusion matrices, and ROC/AUC curves for the intermediate stage.

Cell 14: Fine-tuning setup. Unfreezes the last convolutional block (`layer4`) of ResNet50 and creates a new optimizer (`optimizer_ft`) that updates only parameters with `requires_grad = True`, using a very small learning rate (1×10^{-5}).

Cell 15: Fine-tuning. Fine-tunes the partially unfrozen ResNet50 for another 30 epochs on the FFT dataset, then evaluates on the test set to obtain post-fine-tuning accuracy.

Cell 16: Final detailed evaluation. Runs the fine-tuned model on the test set, generating the final classification report, confusion matrix, and ROC/AUC used in the main results.

Cell 17: Saving the fine-tuned model. Saves the final fine-tuned model weights to Google Drive as `FINAL_finetuned_model_new.pth`, making the best-performing detector available for reuse in later sessions.

Cell 18: Rebuilding and loading the best model. Reconstructs a ResNet50 architecture with a 2-class head, then loads the saved checkpoint corresponding to the best run (`AUC 0.9704 - Run 2 - FINAL_finetuned_model_new.pth`) from Google Drive. Moves the model to the active device and sets it to evaluation mode to prepare for latency testing.

Cell 19: Rebuilding the FFT test loader for latency. Recreates `FFTImageDataset` using `new_dataset_path` and the same ImageNet-style transforms, then performs a fresh 50/25/25 split into train, validation, and test subsets. Constructs a new `test_loader_tl` with batch size 16, which serves as the input source for latency measurements.

Cell 20: Latency measurement. Performs batched inference timing for the final model on the Tesla T4 GPU. The cell draws one batch from `test_loader_tl`, runs several warm-up iterations, and then executes 50 timed forward passes inside a `torch.no_grad()` block. After synchronizing the GPU, it computes the distribution of batch times and reports the median and 95th percentile latency per batch and per image (e.g., median per-image time of approximately 2.7 ms for batch size 16).

E Biographical Note

Joel Villalta-Corcino is a graduate student in the Master of Science in Computer Science program at Central Connecticut State University. His technical focus centers on secure network architecture, virtualization, and identity management. Upon completion of his degree, he aims to advance his career in information security and infrastructure defense.

Christopher Swasey is a graduate student in the Master of Science in Computer Information Technology program at Central Connecticut State University. His academic interests include cybersecurity, digital forensics, and applied machine learning. After graduation, he plans to pursue a career in security engineering and digital investigations.

End of Appendices

References

[1] Gary Bradski. The opencv library. *Dr. Dobb's Journal of Software Tools*, 2000.

[2] Brian Dolhansky, Russell Howes, Ben Pflaum, Nicole Baram, and Cristian Canton Ferrer. The deepfake detection challenge dataset. *arXiv preprint*, 2020.

[3] C. Dong, A. Kumar, and E. Liu. Think twice before detecting GAN-generated fake images from their spectral domain imprints. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[4] R. Durall, M. Keuper, and J. Keuper. Watch your up-convolution: CNN-based generative networks are failing to reproduce spectral distributions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[5] J. Frank, T. Eisenhofer, L. Schönherr, A. Fischer, D. Kolossa, and T. Holz. Leveraging frequency analysis for deep fake image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020.

[6] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, 2014.

[7] Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. *arXiv preprint*, 2015.

[8] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint*, 2015.

[9] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 6840–6851, 2020.

[10] Y. Li and S. Lyu. Exposing deepfake videos by detecting face warping artifacts. In *IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2019.

[11] Yuezun Li, Ming-Ching Chang, and Siwei Lyu. Celeb-DF: A large-scale challenging dataset for deepfake forensics. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[12] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, 2019.

[13] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[14] Ming-Zher Poh, Daniel J. McDuff, and Rosalind W. Picard. Non-contact, automated cardiac pulse measurements using video imaging and blind source separation. *Optics Express*, 18(10):10762–10774, 2010.

[15] Andreas Rössler, Davide Cozzolino, Luisa Verdoliva, Christian Riess, Justus Thies, and Matthias Nießner. Faceforensics++: Learning to detect manipu-

lated facial images. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[16] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[17] C. Tan, Y. Zhao, S. Wei, G. Gu, P. Liu, and Y. Wei. Frequency-aware deepfake detection: Improving generalizability through frequency space learning (FreqNet). In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.